



**UNIVERSITÉ
DE LORRAINE**

Cahier des charges et modélisation

Application Web pour la Gestion de bibliothèques
en React, NestJS, MongoDB et Machine Learning

Groupe 2 :

HAMIDA Rim

RALANTONISAINANA Nyela

Présentation

Ce projet consiste en une application Web de gestion de bibliothèque. Elle repose sur un *frontend* développé en **React**, et sur un *backend* en **NestJS** pour gérer la logique métier, les données et la communication avec la base de données **MongoDB**.

Un module de **Machine Learning** vient enrichir l'application avec deux fonctionnalités principales : La prédiction des livres les plus empruntés afin d'anticiper la popularité et d'optimiser les stocks et la recommandation personnalisée de livres pour chaque utilisateur, grâce à un système de suggestion intelligente.

Le système doit permettre aux membres de consulter le catalogue de livres, de réserver et d'emprunter des ouvrages, tout en offrant des recommandations personnalisées basées sur un module de **Machine Learning**.

Le système doit également permettre aux administrateurs de gérer le catalogue (ajout, modification, suppression de livres), de suivre les emprunts et réservations, et d'anticiper la popularité des ouvrages afin d'optimiser les stocks.

La **prédiction de la popularité des livres**, utilise un modèle de régression/classification pour estimer le nombre d'emprunts futurs par ouvrage (ex. pour le mois prochain).

Fonctionnalités :

- **Utilisateur :**
 - Créer un compte / Se connecter / Se déconnecter ;
 - Modifier son mot de passe oublié ;
 - Supprimer son compte ;
- **Membre :**
 - Modifier les informations de son compte ;
 - Rechercher / Consulter un catalogue (avec filtrage) ;
 - Consulter le détail d'un livre (disponibilité, résumé, évaluations) ;
 - Emprunter un livre ;
 - Réserver un livre ;
 - Consulter les emprunts et réservations en cours ;
- **Personnel :**
 - Ajouter / Modifier / Supprimer des livres
 - Gérer les demandes de réservation des livres
 - Gérer les emprunts des livres
 - Consulter les indicateurs de popularité des livres
- **Machine Learning :**
 - Prédiction de popularité d'un livre (ex : prochain mois).
 - Recommandation de livres aux membres

Objectifs :

Concevoir, développer et déployer cette application Web

Diagramme de classes :

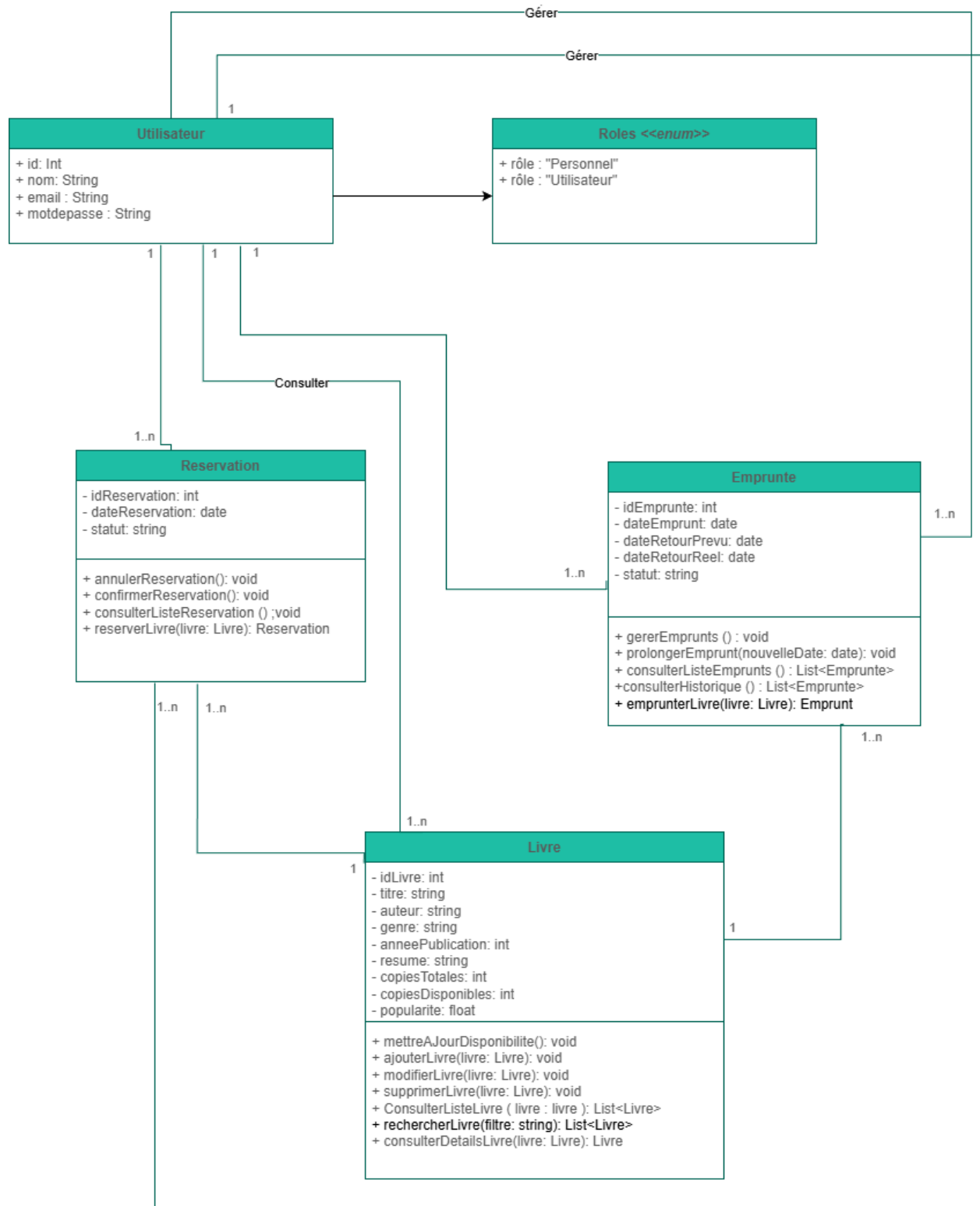
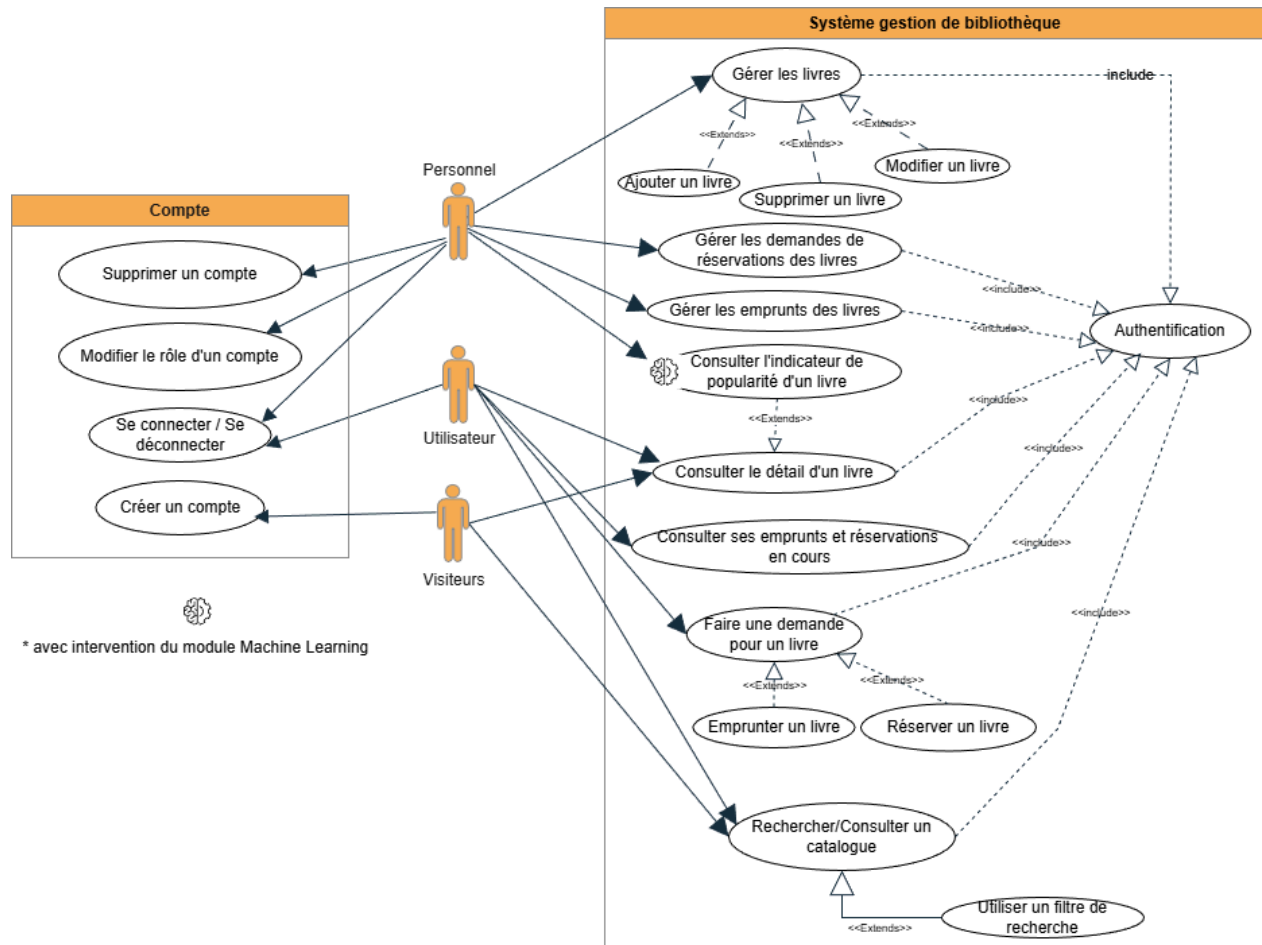


Diagramme de cas d'utilisation :



Présentation du module ML :

Notre système de gestion de bibliothèque ne se limite pas à la gestion des emprunts. Il s'agit d'un **système intelligent** capable d'**analyser les données**, d'**apprendre** des comportements passés et d'**anticiper les besoins futurs**. Grâce au module de **Machine Learning**, le système :

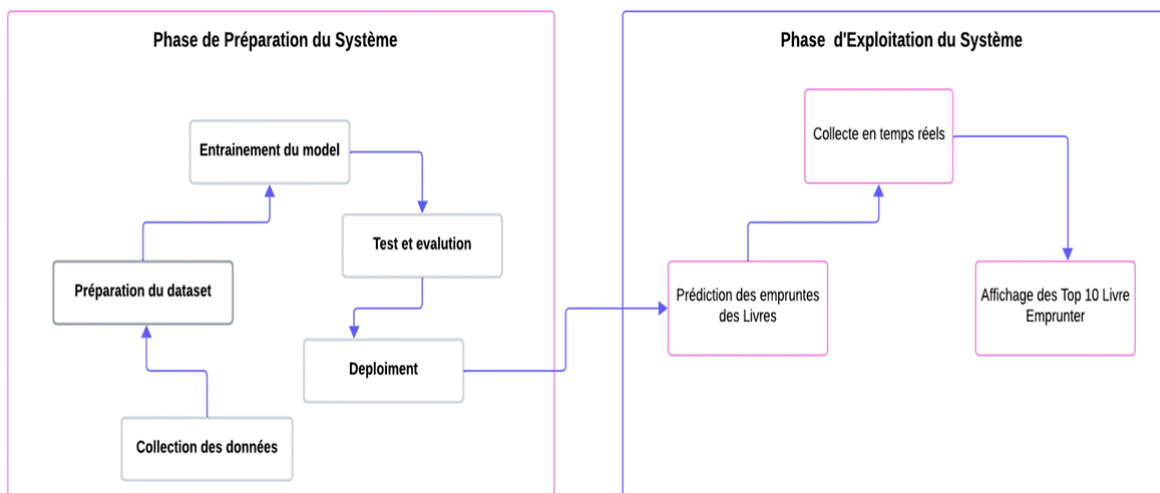
- Prédit les **livres les plus empruntés** pour les mois à venir.
- Affiche un **Top 10 dynamique** des ouvrages les plus populaires.
- Aide à **optimiser les stocks** et à éviter les ruptures.
- Améliorer l'**expérience utilisateur** en rendant la recherche plus pertinente.

Fonctionnement du module ML :

Le module de prédiction a été conçu pour analyser les données d'emprunt et estimer la demande future.

Étapes principales :

1. Préparation et nettoyage des données (auteur, type, exemplaires, localisations).
2. Test de plusieurs modèles de prédiction : **Random Forest** et **Gradient Boosting Regressor**.
3. Sélection du modèle **de Gradient Boosting Regressor**, offrant le meilleur résultat (**R² élevé, RMSE faible**).



4. Déploiement du Modèle : Micro-service FastAPI et Persist

L'intégration du modèle Machine Learning dans l'architecture Full Stack exigeait un découplage technique entre l'écosystème Python (modélisation) et TypeScript/NestJS (backend applicatif). Ce besoin a été résolu par le déploiement d'un micro-service dédié utilisant FastAPI.

5. Architecture et Rôle de FastAPI

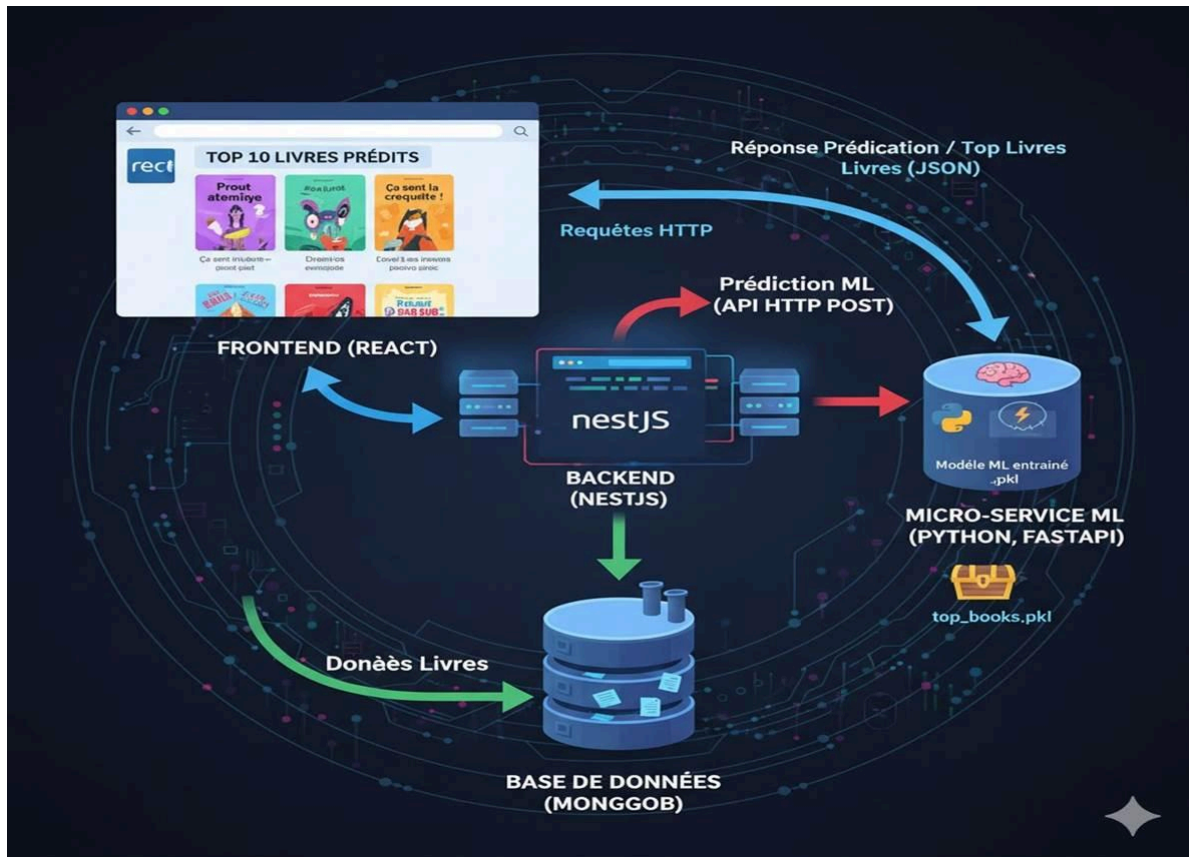
FastAPI sert de **passerelle API RESTful** pour le modèle de régression. Son choix est justifié par sa capacité à gérer des requêtes **asynchrones** et sa compatibilité native avec l'environnement Python.

- **Communication** : Le backend **NestJS** orchestre l'appel en soumettant les données brutes de l'ouvrage (au format JSON) via une requête **HTTP POST** à l'endpoint **/predict_loan** de FastAPI. La prédiction est renvoyée en JSON, permettant à NestJS de l'intégrer en base de données ou de la transmettre directement au **Frontend React** pour un affichage en temps réel.

6. Persistance des Artéfacts avec joblib :

La bibliothèque **joblib** a été mobilisée pour la persistance du modèle (`ml_model.pkl`) et des encodeurs de labels (`label_encoders.pkl`). Nous avons choisi **joblib** car il est **optimisé pour la sérialisation efficace des modèles Scikit-learn** qui manipulent de larges tableaux de données numériques (NumPy arrays), contrairement au `pickle` standard. L'instruction `joblib.load()` est exécutée **une seule fois** au démarrage de FastAPI pour charger les artefacts en mémoire vive.

Voici cette architecture qui représente et explique ce que nous avons implémenté :



Impact du Système Intelligent :

- **Aide à la décision** : les bibliothécaires peuvent anticiper la demande.
- **Automatisation intelligente** : le système apprend et s'améliore avec le temps.
- **Optimisation des ressources** : meilleure gestion du budget et des stocks.
- **Expérience utilisateur enrichie** : recommandations pertinentes et interface plus dynamique.